

操作系统实践

D：从零开始写操作系统

09023213 徐筱睿

项目概况

- 项目背景：参考 uCore Tutorial，构建 OS 内核。
- 开发环境：Ubuntu 22.04 + QEMU 模拟器 + RISC-V GCC 工具链。
- 最终成果
 - 实现了 **LibOS** (裸机运行时环境)。
 - 实现了 **协作式多任务调度** (Multitasking)。
 - 实现了 **虚拟内存管理** (SV39 分页机制)。
 - 实现了 **进程管理** (fork 机制) 与 **交互式 Shell**。

系统架构演进

- 硬件层：RISC-V CPU (QEMU Virt), OpenSBI 固件。
- 内核层 (S-Mode):
 - Trap：中断与异常处理。
 - MM：物理帧分配 + 页表映射。
 - Task：进程控制块(TCB) + 调度器。
- 用户层 (U-Mode):
 - 系统调用库 (lib.c)。
 - Shell (app.c)。

核心模块 I - 启动与特权级隔离

- 裸机启动 (LibOS):
 - 编写 `entry.S` 设置启动栈，清除 BSS 段（防止脏数据导致的随机崩溃）。
 - 封装 OpenSBI 接口 (`ecall`) 实现 `printf` 格式化输出。
- 特权级切换 (Protection):
 - 用户态 (U-Mode)：应用程序运行受限，不能直接访问硬件。
 - 内核态 (S-Mode)：处理系统调用与异常。
 - Trap 上下文：实现 `__alltraps` 和 `__restore`，在栈上保存/恢复 32 个通用寄存器。

核心模块 III - 内存管理

- 物理内存管理 (Frame Allocator):
 - 实现栈式分配算法，动态管理 `0x80000000 ~ 0x88000000` (128MB) 空间。
- 虚拟内存 (Virtual Memory):
 - 启用 RISC-V SV39 分页机制 (`satp` 寄存器)。
 - 内核映射：实现恒等映射 (Identity Mapping)，确保开启分页瞬间内核不崩溃。
 - 用户映射：实现 `uvm_map`，将用户代码和栈映射到固定的虚拟地址。
 - 成果：实现了地址空间隔离，不同进程拥有独立的页表。

核心模块 II - 多道程序与调度

- 任务抽象：
 - 设计 `TaskControlBlock` (TCB)，包含内核栈、用户栈、任务上下文。
- 上下文切换 (Context Switch):
 - 实现 `__switch` 汇编函数，在两个内核线程之间切换 Callee-Saved 寄存器。
 - 调度策略：实现协作式调度 (`sys_yield`)，任务主动让出 CPU，内核轮询寻找下一个就绪任务 (Round Robin)。

核心模块 IV - 进程管理与 Shell

- 进程分叉 (`sys_fork`):
 - 实现深拷贝：遍历父进程页表，为子进程分配新物理页并复制数据 (`uvm_copy`)。
 - Trap 上下文复制：子进程继承父进程的寄存器状态，仅修改返回值 (`a0=0`)。
- 交互式 Shell：
 - 实现 `sys_read` 系统调用，通过 SBI 获取键盘输入。
 - 编写行编辑器：支持回显、退格修改、回车执行。
 - 最终成果：Shell 解析 `test` 命令 -> fork 子进程 -> 父子进程交替并发运行。

报错解决

```
Platform Name      : riscv-virtio,qemu
Platform Features  : timer,mfdeleg
Platform HART Count : 1
Firmware Base     : 0x80000000
Firmware Size     : 100 KB
Runtime SBI Version : 0.2
Domain0 Name      : root
Domain0 Boot HART  : 0
Domain0 HARTs     : 0*
Domain0 Region00   : 0x0000000080000000-0x000000008001ffff ( )
Domain0 Region01   : 0x0000000000000000-0xffffffffffffff (R,W,X)
Domain0 Next Address : 0x0000000000000000
Domain0 Next Arg1   : 0x00000000087000000
Domain0 Next Mode   : S-mode
Domain0 SysReset    : yes
Boot HART ID       : 0
Boot HART Domain    : root
Boot HART ISA       : rv64imafdcsu
Boot HART Features  : scounteren,mcounteren,time
Boot HART PMP Count : 16
Boot HART PMP Granularity : 4
Boot HART PMP Address Bits: 54
Boot HART MHPM Count : 0
Boot HART MHPM Count : 0
Boot HART MIDELEG   : 0x0000000000000222
Boot HART MEDELEG   : 0x000000000000b109
```

内核代码成功加载到内存 0x80200000

OpenSBI固件启动后

但是错误跳转到 0x0000000000000000

CPU跳转到0地址（虚空），而不是写好的代码地址

解决：

修改 QEMU 的启动参数，使QEMU自动处理跳转地址

Makefile 中：

QEMU_OPTS := -machine virt -nographic

-bios default

-kernel kernel.elf

报错解决 内核在处理系统调用时崩溃

```
Boot HART ID       : 0
Boot HART Domain    : root
Boot HART ISA       : rv64imafdcsu
Boot HART Features  : scounteren,mcounteren,time
Boot HART PMP Count : 16
Boot HART PMP Granularity : 4
Boot HART PMP Address Bits: 54
Boot HART MHPM Count : 0
Boot HART MHPM Count : 0
Boot HART MIDELEG   : 0x0000000000000222
Boot HART MEDELEG   : 0x000000000000b109
```

```
[ToyOS] Phase 3: Privilege Switching
[Kernel] App loaded. Preparing to switch to User Mode...
[Kernel] Jumping to User App via __restore!
[Kernel] Trap! scause=8 stval=0 sepc=80400034
[Kernel] Trap! scause=5 stval=48 sepc=802000d4
[Kernel] Fatal Exception! scause=5, stval=48, sepc=802000d4
```

处理方式:

- 展开宏，手动写32个寄存器

- 在 trap.c 中添加调试语句

检查 buf 指针内容

`printf("[Kernel] sys_write: fd=%d, buf=%x, len=%d\n", fd, buf, len);`

```
[ToyOS] Phase 3: Privilege Switching
[Kernel] App loaded. Preparing to switch to User Mode...
[Kernel] Jumping to User App via __restore!
[Kernel] Trap! scause=8 stval=0 sepc=80400034
[Kernel] sys_write: fd=1, buf=80400088, len=44
[Kernel] Trap! scause=5 stval=48 sepc=802000d4
[Kernel] Fatal Exception! scause=5, stval=48, sepc=802000d4
```

Sys_write 打印了参数，说明系统调用进去了

接着发生 scause=5且stval=48，说明崩溃发生在系统调用返回之后的恢复现场 __restore 时

报错解决 内核在处理系统调用时崩溃

Stval = 48 十六进制 0x30 说明

程序试图从地址 0x30 读取数据，而该地址未映射或不可访问

在__restore的汇编中

```
ld x6, 6*8(sp) # 6*8 = 48
```

从 sp+48 处加载数据到 x6

如果 sp 变成了 0，那么这行代码就是在读取内存地址 48，从而触发 Load Access Fault (scause=5, stval=48)

trap_handler 函数在返回给汇编代码时，把栈指针弄丢了，返回了 0 (NULL)。这通常是因为编译器优化 (-O2) 导致的调用约定问题。

解决方法: 强制传递上下文指针

让syscall 也返回 TrapContext，保持数据流连贯

报错解决 内核主线程和Task0混淆

```
[ToyOS] Phase 4: Multiprogramming
[Kernel] Initializing tasks...
[Kernel] Task 0 created.
[Kernel] Task 1 created.
[Kernel] Task 2 created.
[Kernel] Starting schedule...
[Kernel] Switch 0 → 1
Hello, I am a Task!
I am yielding now... (1/3)
[Kernel] Switch 1 → 2
Hello, I am a Task!
I am yielding now... (1/3)
[Kernel] Switch 2 → 0
```

第一次调度(Switch 0 → 1)

main函数调用 schedule()

此时 current_task_id = 0

调度器切换到 Task 1

执行 __switch(&tasks[0].context, &tasks[1].context)

把当前 main 函数的内核上下文保存到了 tasks[0].context, 导致 Task0变成了内核主线程

Task 1 → 2: 正常切换

Task 2 → 0: 切换回 Task 0

但是此时 Task 0 被覆盖成了 main， 所以这里变成了死循环，系统停止

报错解决 内核主线程和Task0混淆

解决方法:

os/task.c 中

新增一个 TaskContext idle_cx 专门用来保存内核主线程(main的上下文)

当前任务 ID 初始化为 -1, 表示没有任何用户任务在跑

修改第一次调度的特殊逻辑 (如右图)

```
if (current_task_id == -1) {
    next_id = 0; // 从 Task 0 开始跑
} else {
    next_id = (current_task_id + 1) % app_num;
    while (tasks[next_id].is_running == 0) {
        next_id = (next_id + 1) % app_num;
        if (next_id == current_task_id) {
            printf("[Kernel] All tasks finished!\n");
            while(1);
        }
    }
}
```

报错解决 栈溢出踩踏

```
[ToyOS] Phase 4: Multiprogramming
[Kernel] Initializing tasks...
[Kernel] Task 0 created.
[Kernel] Task 1 created.
[Kernel] Task 2 created.
[Kernel] Starting schedule...
[Kernel] Idle → Task 0
Hello, I am a Task!
I am yielding now... (1/3)
[Kernel] Switch 0 → 1
Hello, I am a Task!
I am yielding now... (1/3)
[Kernel] Switch 1 → 2
Hello, I am a Task!
I am yielding now... (1/3)
[Kernel] All tasks finished!
```

内存布局

TaskControlBlock 结构体中, kernel_stack放在最前面, is_running放在最后。

在tasks数组中, Task0 和 Task1 紧挨着, Task1 的内存地址紧挨着 Task0。

RISC-V 中 栈是向下生长的:

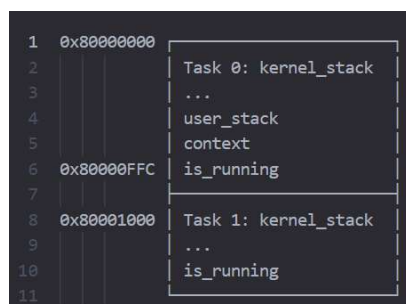
1	0x80000000	Task 0: kernel_stack	← 栈顶初始位置 (高地址 within Task 0)
2		...	
3		user_stack	
4		context	
5		is_running	← Task 0 的**尾部**
6	0x8000FFC		← Task 1 开始 (0x80001000)
7		Task 1: kernel_stack	← Task 1 的栈顶
8	0x80001000	...	
9		is_running	← Task 1 的尾部
10			
11			

报错解决 栈溢出踩踏

当 Task 1 的kernel_stack 使用过多，sp会继续向下减小，当 $sp < 0x80001000$ 时，超出了 Task 1 的栈范围，越界写入 Task 0 的区域。

而Task 0 的高位是 is_running，会被覆盖。

Task 1 的is_running 同理，可能被 Task 2 的溢出覆盖。



解决方法: 修改栈顺序

```
// // 2. 定义任务控制块
// typedef struct {
//     uint64_t kernel_stack[4096 / 8]; // 内核栈
//     uint64_t user_stack[4096 / 8];   // 用户栈
//     TaskContext context;              // 切换上下文
//     int is_running;                   // 状态
// } TaskControlBlock;

// 修改栈顺序
typedef struct {
    int is_running;                    // 状态
    TaskContext context;              // 切换上下文
    uint64_t user_stack[4096 / 8];   // 用户栈
    uint64_t kernel_stack[4096 / 8]; // 内核栈
} TaskControlBlock;
```

报错解决

Scause: Store Access Fault 写内存错误

Stval 为负数

意味着栈指针sp变成0，CPU试图向sp - 264 的位置写数据，但是地址是非法地址。

修改：

在kernel.ld 中手动标记出 .bss 段的开始(sbss) 和结束(ebss)，

在entry.S 设置栈指针之前，用一个循环手动把sbss到ebss之间的内存全部写0

```
xuxiaorui@xuxiaorui-virtual-machine:~/os-project$ make run

Platform Name      : riscv-virtio,qemu
Platform Features  : timer,mfdeleg
Platform HART Count : 1
Firmware Base      : 0x80000000
Firmware Size      : 100 KB
Runtime SBI Version : 0.2

Domain0 Name       : root
Domain0 Boot HART  : 0
Domain0 HARTs      : 0*
Domain0 Region00   : 0x0000000080000000-0x000000008001ffff ()
Domain0 Region01   : 0x0000000000000000-0xffffffffffff (R,W,X)
Domain0 Next Address : 0x0000000080200000
Domain0 Next Arg1   : 0x0000000087000000
Domain0 Next Mode    : S-mode
Domain0 SysReset    : yes

Boot HART ID       : 0
Boot HART Domain    : root
Boot HART ISA       : rv64imafdcsv
Boot HART Features  : scounteren,mcounteren,time
Boot HART PMP Count : 16
Boot HART PMP Granularity : 4
Boot HART PMP Address Bits: 54
Boot HART MHPM Count : 0
Boot HART MHPM Count : 0
Boot HART MIDELEG   : 0x000000000000222
Boot HART MEDELEG   : 0x000000000000b109

[ToyOS] Phase 5: Memory Management
[Kernel] Memory Manager Initialized.
[Kernel] Free RAM start: 8025d000, end: 80000000
[Kernel] Fatal Exception! scause=7, stval=fffffef8, sepc=8020029a
```


报错解决 无限重启

无限循环输出左图，调试输出结果如右图

Fence.o 刷新指令缓存，防止 CPU 读到旧数据

```
[ToyOS] Phase 6: Page Table Mapping
[Kernel] Checking BSS: recycled_ptr=0 (Expect 0)
[Kernel] Memory Manager Initialized.
[Kernel] Free RAM start: 80220000, end: 88000000
[Kernel] Allocated addr: 80220000
[Kernel] Kernel PT created at 80220000
[Kernel] Allocated addr: 80221000
[Kernel] Allocated addr: 80222000
[Kernel] Map UART... done.
[Kernel] Allocated addr: 80223000
[Kernel] Allocated addr: 80224000

[ToyOS] Phase 6: Page Table Mapping
[Kernel] Checking BSS: recycled_ptr=0 (Expect 0)
[Kernel] Memory Manager Initialized.
[Kernel] Free RAM start: 80220000, end: 88000000
[Kernel] Allocated addr: 80220000
[Kernel] Kernel PT created at 80220000
[Kernel] Allocated addr: 80221000
[Kernel] Allocated addr: 80222000
[Kernel] Map UART... done.
[Kernel] Allocated addr: 80223000
[Kernel] Allocated addr: 80224000

[ToyOS] Phase 6: Page Table Mapping
[Kernel] Checking BSS: recycled_ptr=0 (Expect 0)
[Kernel] Memory Manager Initialized.
[Kernel] Free RAM start: 80220000, end: 88000000
[Kernel] Allocated addr: 80220000
[Kernel] Kernel PT created at 80220000
[Kernel] Allocated addr: 80221000
[Kernel] Allocated addr: 80222000
[Kernel] Map UART... done.
[Kernel] Allocated addr: 80223000
[Kernel] Allocated addr: 80224000
```

```
[Kernel] Allocated addr: 80261000
[Kernel] Allocated addr: 80262000
[Kernel] Allocated addr: 80263000
[Kernel] Map Data/BSS/Heap... done.
[Kernel] Paging ENABLED! Hello from Virtual World!
[Kernel] System matches Physical Memory 1:1.
[Kernel] Initializing tasks with Virtual Memory...
[Kernel] Allocated addr: 80264000
[Kernel] Allocated addr: 80265000
[Kernel] Allocated addr: 80266000
[Kernel] Task 0 created. PT=80264000
[Kernel] Allocated addr: 80267000
[Kernel] Allocated addr: 80268000
[Kernel] Task 1 created. PT=80267000
[Kernel] Allocated addr: 80269000
[Kernel] Allocated addr: 8026a000
[Kernel] Task 2 created. PT=80269000
[Kernel] Idle -> Task 0

[ToyOS] Phase 6: Page Table Mapping
[Kernel] Checking BSS: recycled_ptr=0 (Expect 0)
[Kernel] Memory Manager Initialized.
[Kernel] Free RAM start: 80220000, end: 88000000
[Kernel] Allocated addr: 80220000
[Kernel] stext=80200000, etext=80201000
[Kernel] Text Size=1000
[Kernel] Map UART... done.
[Kernel] Start mapping Text...
Mapping VA 80200000
Mapping VA 80201000
[Kernel] Map Text... done.
[Kernel] Map Rodata... done.
[Kernel] Map Data/BSS/Heap... done.
[Kernel] Paging ENABLED! Hello from Virtual World!
[Kernel] System matches Physical Memory 1:1.
[Kernel] Initializing tasks with Virtual Memory...
[Kernel] Allocated addr: 80221000
[Kernel] Allocated addr: 80222000
[Kernel] Task 0 created. PT=80221000
[Kernel] Allocated addr: 80223000
[Kernel] Allocated addr: 80224000
```

报错解决 子进程不运行

• 现象

- 父进程成功执行了fork，子进程被创建且在tasks数组中。
- 父进程调用yield，希望把CPU让给别人，但是调度器遍历一遍后，把CPU又还给父进程了。
- 父进程(Task 0)退出后，调度器宣布所有任务完成，无视 Task 1。

• 解决

- 调度器遍历整个任务池，不论设定的app_num 是多少

```
[ToyOS] Phase 6: Page Table Mapping
[Kernel] Checking BSS: recycled_ptr=0 (Expect 0)
[Kernel] Memory Manager Initialized.
[Kernel] Free RAM start: 8021c000, end: 88000000
[Kernel] stext=80200000, etext=80201000
[Kernel] Text Size=1000
[Kernel] Map UART... done.
[Kernel] Start mapping Text...
Mapping VA 80200000
Mapping VA 80201000
[Kernel] Map Text... done.
[Kernel] Map Rodata... done.
[Kernel] Map Data/BSS/Heap... done.
[Kernel] Paging ENABLED! Hello from Virtual World!
[Kernel] System matches Physical Memory 1:1.
[Kernel] Initializing tasks with Virtual Memory...
[Kernel] Task 0 created. PT=80260000
[Kernel] Idle -> Task 0
I am the Parent. Calling fork()...
[Parent] Fork success! Child PID = (child_pid)
[Parent] I will wait (yield) for child.
[Parent] I am back. Bye!
[Kernel] App 0 exited.
[Kernel] All tasks finished!
```

报错解决 无限克隆

• 现象

- Task 0 Fork Task 1
- Task 1 并没有认为自己是子进程，而是认为自己也是父进程，于是又 Fork 了 Task 2
- Task 2 同理
- 直到任务槽位满，发生 Panic

• 原因

- 子进程没有经过 syscall 函数的末尾，所以它的 sepc 依然指向 ecall，相当于又调用一次 sys_fork

```
[ToyOS] Phase 6: Page Table Mapping
[Kernel] Checking BSS: recycled_ptr=0 (Expect 0)
[Kernel] Memory Manager Initialized.
[Kernel] Free RAM start: 8021c000, end: 88000000
[Kernel] stext=80200000, etext=80201000
[Kernel] Text Size=1000
[Kernel] Map UART... done.
[Kernel] Start mapping Text...
Mapping VA 80200000
Mapping VA 80201000
[Kernel] Map Text... done.
[Kernel] Map Rodata... done.
[Kernel] Map Data/BSS/Heap... done.
[Kernel] Paging ENABLED! Hello from Virtual World!
[Kernel] System matches Physical Memory 1:1.
[Kernel] Initializing tasks with Virtual Memory...
[Kernel] Task 0 created. PT=80260000
[Kernel] Idle → Task 0
I am the Parent. Calling fork()...
[Parent] Fork success! Child PID = (child_pid)
[Parent] I will wait (yield) for child.
[Parent] Fork success! Child PID = (child_pid)
[Parent] I will wait (yield) for child.
[Parent] Fork success! Child PID = (child_pid)
[Parent] I will wait (yield) for child.
[Kernel] No free task slot for fork!
[Parent] Fork failed!

[Kernel] PANIC! Exception @ Kernel Mode
scause = 12 (Exception Type)
stval = 0 (Bad Address)
sepc = 0 (Instruction Address)
```

展望

- 引入文件系统以支持 exec
- 当前是协作式，依赖 App 主动调用 yield
- 需要引入时钟中断实现 抢占式分时系统